



**COMSATS University Islamabad,
Abbottabad Campus**

SOFTWARE DESIGN DESCRIPTION

(SDD DOCUMENT)

for

INTELI-REHAB

A Smart Wearable Rehabilitation System

By

Tehreem Zaheer	CIIT/SP23-BCS-131/ATD
Kashmala Zeb	CIIT/SP23-BSE-048/ATD
Ahmed Irshad	CIIT/SP23-BSE-045/ATD

Supervisor

Dr. Ahmed Khan

Bachelor of Science in Computer Science (2023-2027)

Table of Contents

1. Introduction.....	6
2. Design methodology and software process model	6
3. System overview	7
3.1 Architectural design	7
3.2 ARCHITECTURE DIAGRAM:.....	9
3.3 Process flow/Representation	9
3.3.1 Patient Activity Diagram:	10
3.3.2 Physiotherapist Activity Diagram:.....	11
3.3.3 Clinic Activity Diagram:.....	12
4. Design models	13
4.1 Class Diagram.....	13
4.2 Sequence Diagram	14
Patient Sequence Diagram	14
Physiotherapist Sequence Diagram	15
Clinic Sequence Diagram	16
7. Data design	18
5.2 DATABASE / STORAGE SOLUTIONS:.....	18
5.3 DATA DICTIONARY:.....	19
8. ALGORITHM AND IMPLEMENTATION:.....	26
6.1 WEARABLE PAIRING & BIOMETRIC CALIBRATION ALGORITHM:	26
6.2 SENSOR FUSION ALGORITHM:.....	27
6.3 EMG SIGNAL PROCESSING ALGORITHM:.....	28
6.4 MOVEMENT ANOMALY DETECTION ALGORITHM:.....	28
6.5 FATIGUE TREND DETECTION & ALERT ALGORITHM:.....	29
6.6 SESSION REPORT GENERATION & CLOUD SYNC ALGORITHM:	30
7. Software requirements traceability matrix	31
8. Human interface design	33
8.1. SCREEN IMAGES FOR MOBILE APP:.....	33
The Inteli-Rehab system includes:	33

Screen objects and actions for Patient Mobile App	36
8.2. SCREEN IMAGES FOR WEB PORTAL:.....	37
9. Appendix I.....	40

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken
Comments for 10%: Task list for 30%: Verbally communicated as mentioned in presentation.	We have implemented UI for all three actors (Patient, Physiotherapist and Clinic) with user authentication and constructed the wearable prototype.

Supervised by

Dr. Ahmad Khan

Signature_____

1. Introduction

IntelliRehab is a smart wearable rehabilitation system designed to assist patients and physiotherapists in monitoring and improving upper-arm rehabilitation exercises. The system combines wearable sensor technology, a mobile application, AI-based movement analysis, and 3D digital twin visualization to provide an interactive and data-driven rehabilitation experience.

The system allows patients to perform assigned exercises while collecting real-time motion and muscle activity data through IMU and EMG sensors integrated with an ESP32-based wearable device. The collected data is processed to evaluate movement quality, detect abnormalities, monitor fatigue, and provide corrective feedback. Physiotherapists can manage patients, assign exercises, review progress, and monitor rehabilitation outcomes through a dedicated control interface.

This document presents the system design, architecture, major workflows, database structure, and interface designs completed so far. At the current stage, the system includes modules for patient and clinic onboarding, exercise management, wearable sensor connectivity, baseline calibration, movement tracking, digital twin visualization, AI-based assessment, and physiotherapist monitoring.

2. Design methodology and software process model

The design of Intelli-Rehab follows an Object-Oriented Design (OOD) methodology. This approach is selected because the system contains multiple modular components -- Onboarding, Exercise Protocol Management, Wearable Connectivity, Biometric Calibration, Movement Tracking, Digital Twin Visualization, AI Monitoring & Safety, Gamification, and Physiotherapist/Clinic Oversight -- each of which can be modelled as an independent class or service with clear responsibilities and well-defined interfaces.

Object-oriented principles such as encapsulation, modularity, inheritance, and reusability help ensure that the Flutter mobile application, the on-device intelligence layer, and the Supabase-backed cloud services maintain a clean, testable, and scalable structure. Shared behaviour between Patient, Physiotherapist, and Clinic Admin accounts is captured through a common User base class, while specialised responsibilities are pushed into their respective subclasses.

Intelli-Rehab is developed using the Incremental Development Model, matching the phased delivery plan set out in the project's Scope Document and Gantt chart (10%, 30%, 60%, and 100% milestones). Each increment delivers a working, demonstrable subset of the system -- starting with onboarding and wearable connectivity, then movement tracking and the digital twin, then AI monitoring and safety features, and finally the physiotherapist/clinic dashboards and cloud synchronisation -- allowing the design to be refined and validated against supervisor and committee feedback at every phase.

3. System overview

Inteli-Rehab is designed as a hybrid edge-cloud system that follows a layered, modular architecture. Time-critical sensing and feedback -- sensor fusion, EMG processing, AI inference, and 3D digital twin rendering -- run on-device inside the Flutter mobile application so that patients receive corrective and fatigue feedback with minimal latency during a live session. All clinical oversight, protocol management, and longitudinal progress data are synchronised to a Supabase cloud backend (PostgreSQL) and exposed to physiotherapists and clinic administrators through dedicated React.js web dashboards.

The system operates in an offline-first mode for the exercise session itself -- sensor streaming, movement tracking, and alerting continue to function without connectivity -- while account management, protocol assignment, and reporting require the mobile app or web dashboard to reach the cloud backend. Supabase (PostgreSQL) is used as the primary relational data store for all structured clinical and account data.

3.1 Architectural design

3.1.1 PERCEPTION / SENSING LAYER:

The hardware layer with dual IMU sensors (upper arm + forearm) and EMG electrodes (biceps + triceps). A BLE microcontroller streams raw sensor data to the mobile app.

3.1.2 COMMUNICATION LAYER:

Handles Bluetooth connection between wearable and mobile app. Manages pairing, data streaming, and sensor calibration.

3.1.3 INTELLIGENCE & PROCESSING LAYER:

Processes sensor data on the phone in real time:

- **Sensor Fusion:** Combines IMU data into joint angles
- **EMG Processor:** Calculates muscle activation and fatigue
- **AI Engine:** Detects movement errors and fatigue trends
- **Digital Twin:** Shows a live 3D arm model with color-coded feedback

3.1.4 APPLICATION LAYER:

The Flutter mobile app patients use. Handles onboarding, device pairing, live exercises, progress tracking, and gamified feedback.

3.1.5 DATA & SYNC LAYER:

Stores session data locally on the phone and syncs to the cloud when internet is available.

3.1.6 CLOUD & INFRASTRUCTURE LAYER:

Supabase backend for user login, database, live updates, and file storage.

3.1.7 PRESENTATION / MONITORING LAYER:

React.js web dashboards for physiotherapists (patient monitoring, reports) and clinic admins (staff management).

3.1.8 AI TRAINING PIPELINE:

Offline Python system that trains AI models on EMG datasets, then exports them to the mobile app.

3.1.9 INTER-LAYER RELATIONSHIPS:

- Wearable sends data → Mobile App
- Mobile App processes → AI Engine
- AI results → Live 3D model + alerts
- App saves data → Local storage → Cloud
- Cloud feeds → Web dashboards
- Training updates models → Mobile App (via app updates)

3.2 ARCHITECTURE DIAGRAM:

Figure 1 shows the full layered architecture of Intelli-Rehab, from the wearable hardware through on-device intelligence to the cloud backend and web dashboards.

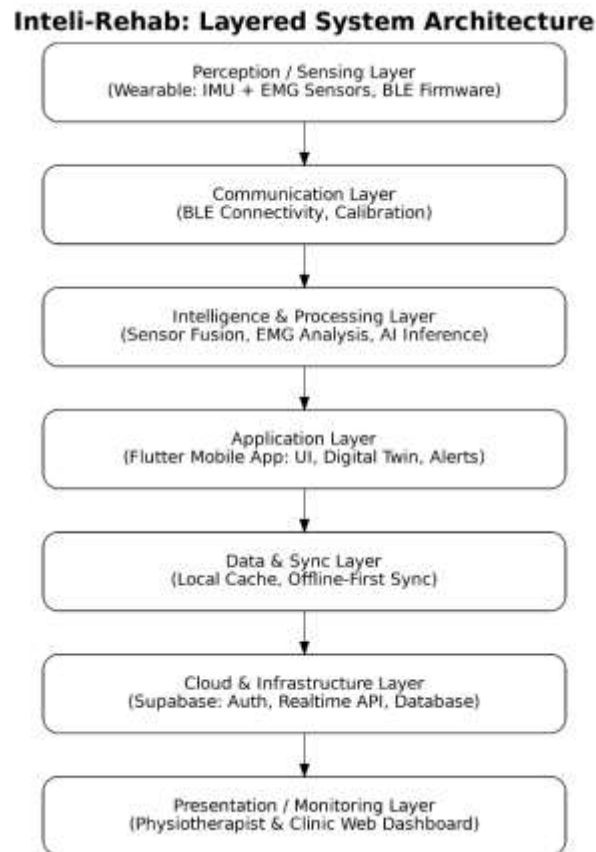


Figure 1 Architecture Diagram

3.3 Process flow/Representation

The following activity diagrams show the process flow of our system based upon three user perspectives, as given below:

- Patient Activity Diagram
- Physiotherapist Activity Diagram
- Clinic Activity Diagram

3.3.1 Patient Activity Diagram:

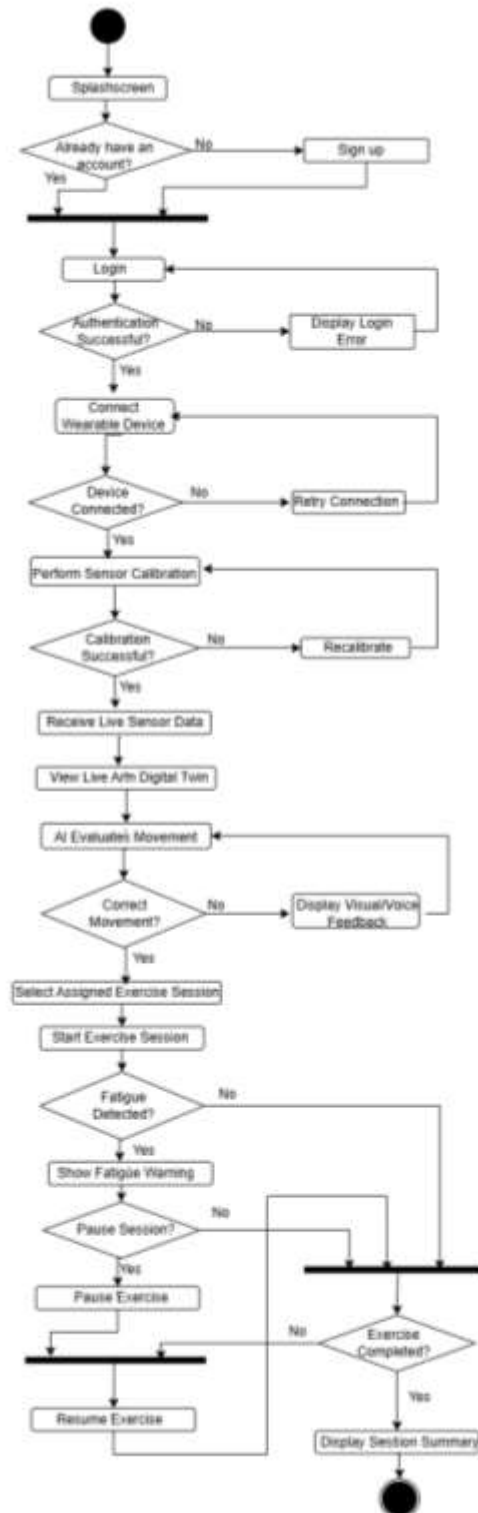


Figure 2 Patient Activity Diagram

3.3.2 Physiotherapist Activity Diagram:

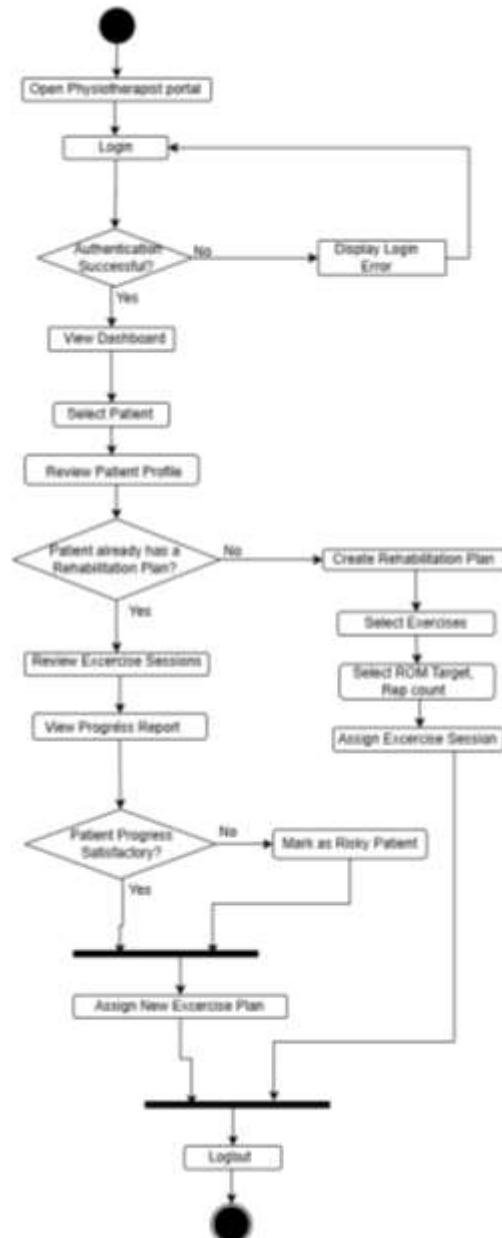
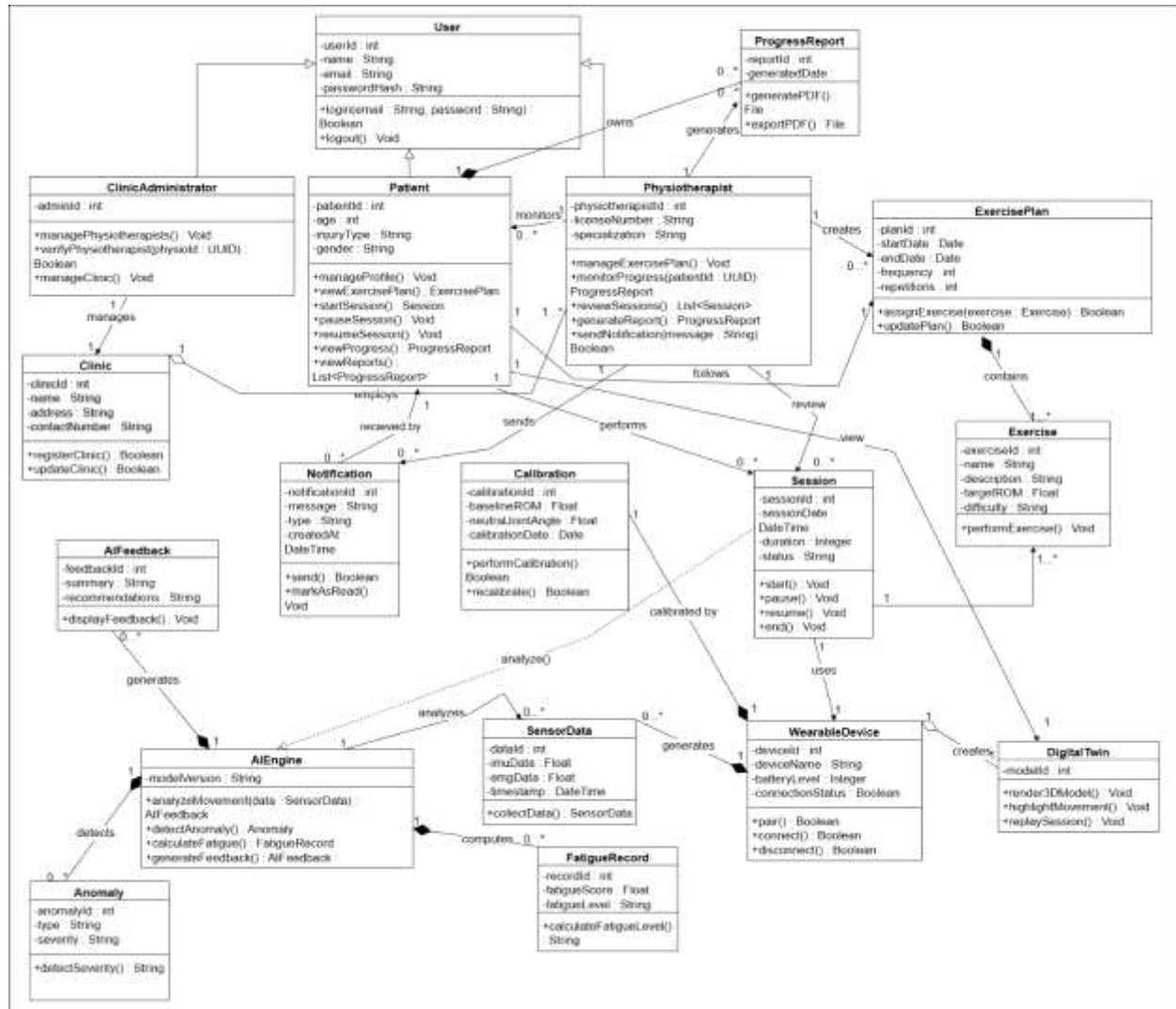


Figure 3 Physiotherapist Activity Diagram

3.3.3 Clinic Activity Diagram:



Figure 4 Clinic Activity Diagram



4.2 Sequence Diagram

The following diagrams show the process sequence of our system based upon three user perspectives, as given below:

Patient Sequence Diagram

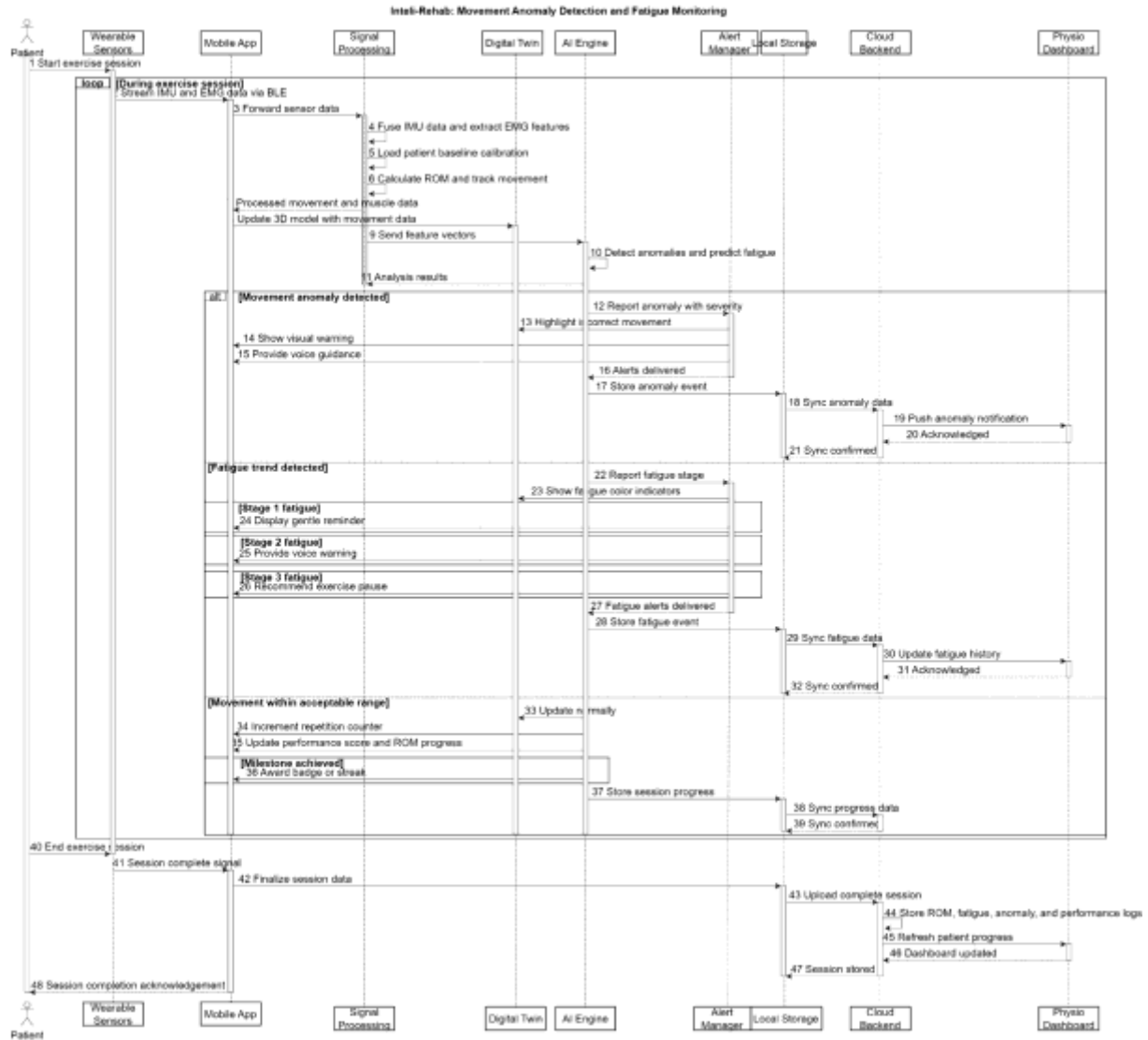


Figure 6 Patient Sequence Diagram

Physiotherapist Sequence Diagram

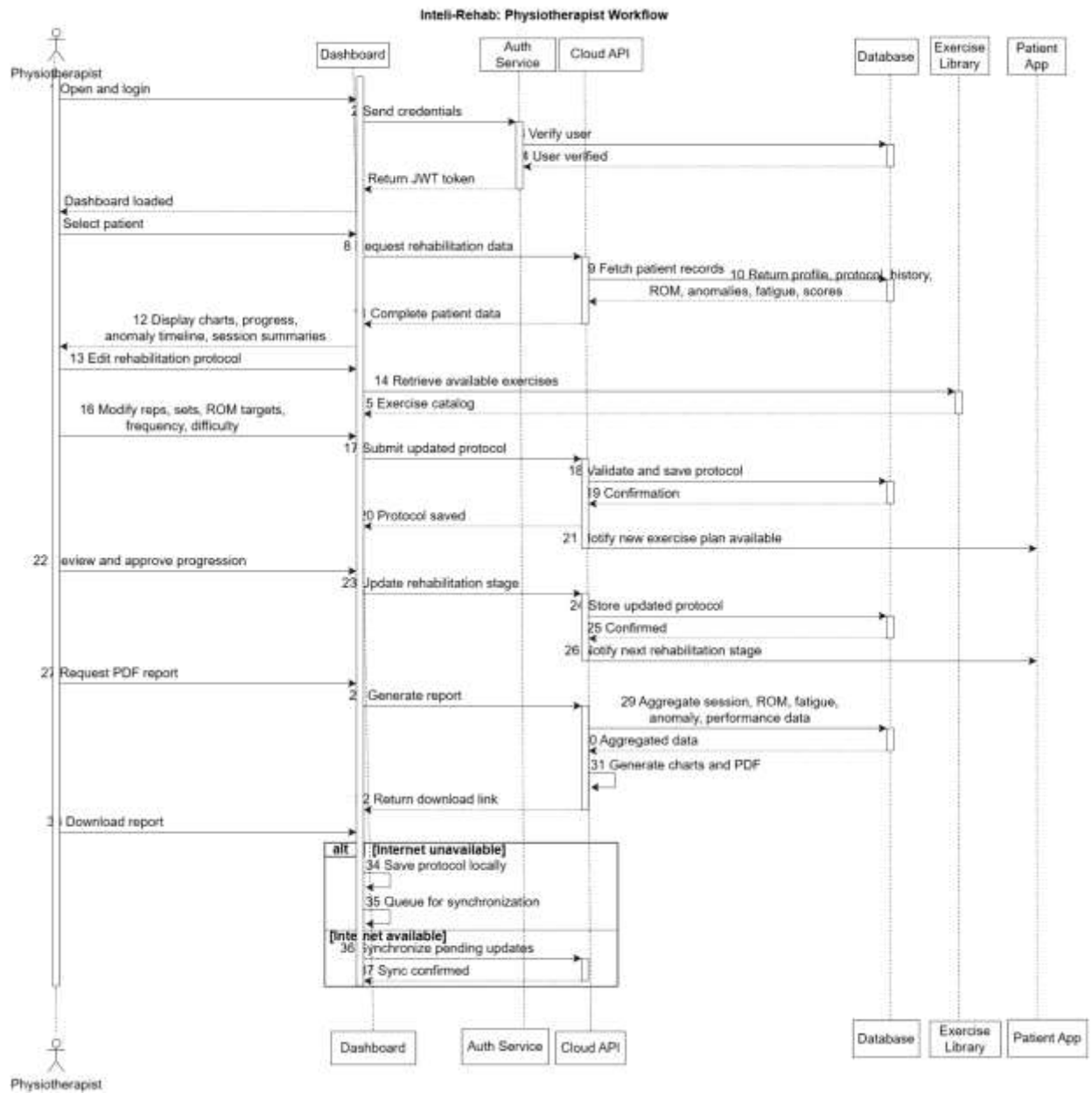


Figure 7 Physiotherapist Sequence Diagram

Clinic Sequence Diagram

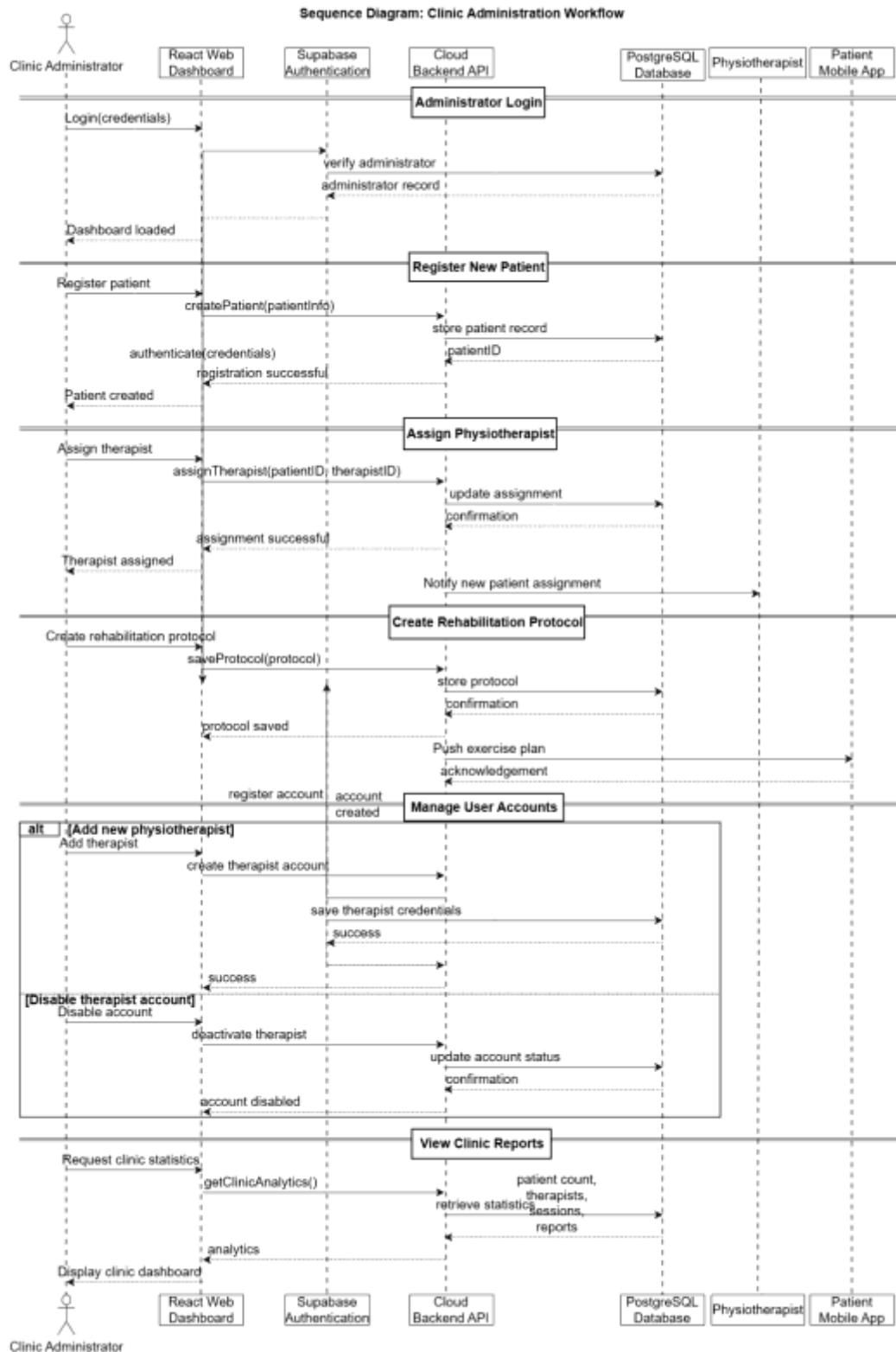


Figure 8 Clinic Sequence Diagram

5. Schematic Diagram:

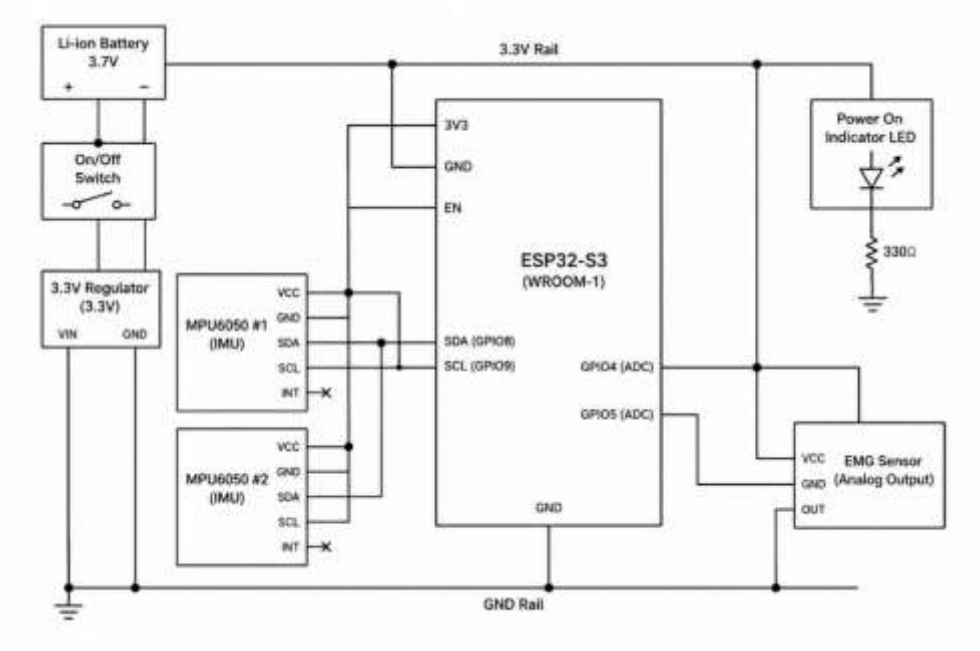


Figure 9 Schematic Block Diagram

6. Timing Diagram

Object / lifeline	T0 Start	T1 Device ready	T2 Login / clinic	T3 Connect / calibrate	T4 Load plan	T5 Exercise	T6 Feedback	T7 Sync	T8 Result
Patient	Start app	Power on wearable	Login / select clinic	Connect & calibrate	Select exercise plan	Perform exercise		Sync	View progress
Mobile app		Close / idle	Authenticate user	BLE pairing + calibration	Load exercise plan	Stream sensor data + render digital twin		Sync session data	Display progress report
Wearable device (ESP32 + IMU + EMG)		Boot + init sensors	Idle / advertising	BLE pairing + baseline capture	Standby	Stream IMU / EMG data		Continue streaming	Idle
On-device AI (fusion, autoencoder, LSTM)		Inactive		Compute baseline	Load ROM targets	Joint angle + EMG processing	Anomaly + fatigue detection	Generate alerts	
Digital twin renderer		Inactive				Render live 3D model		Freeze / replay	
Cloud backend (Supabase / PostgreSQL)		Ready	Auth check	Store device pairing	Provide exercise plan	Idle (local processing)		Receive sync + alerts	Update progress records
Physiotherapist dashboard								Receive alert notification	Review session report
Main events	Start	Ready	Login/ select	Connect/ calibrate	Load plan	Exercise	Feedback	Sync	Result

Figure 10 Timing Diagram

7. Data design

Inteli-Rehab's data design transforms the core information domain -- accounts, clinics, exercise protocols, live sessions, sensor readings, alerts, progress, and reports -- into a well-structured, scalable set of relational entities hosted in Supabase (PostgreSQL). The design supports offline-first session capture, clinician oversight, AI analysis logging, and patient motivation tracking.

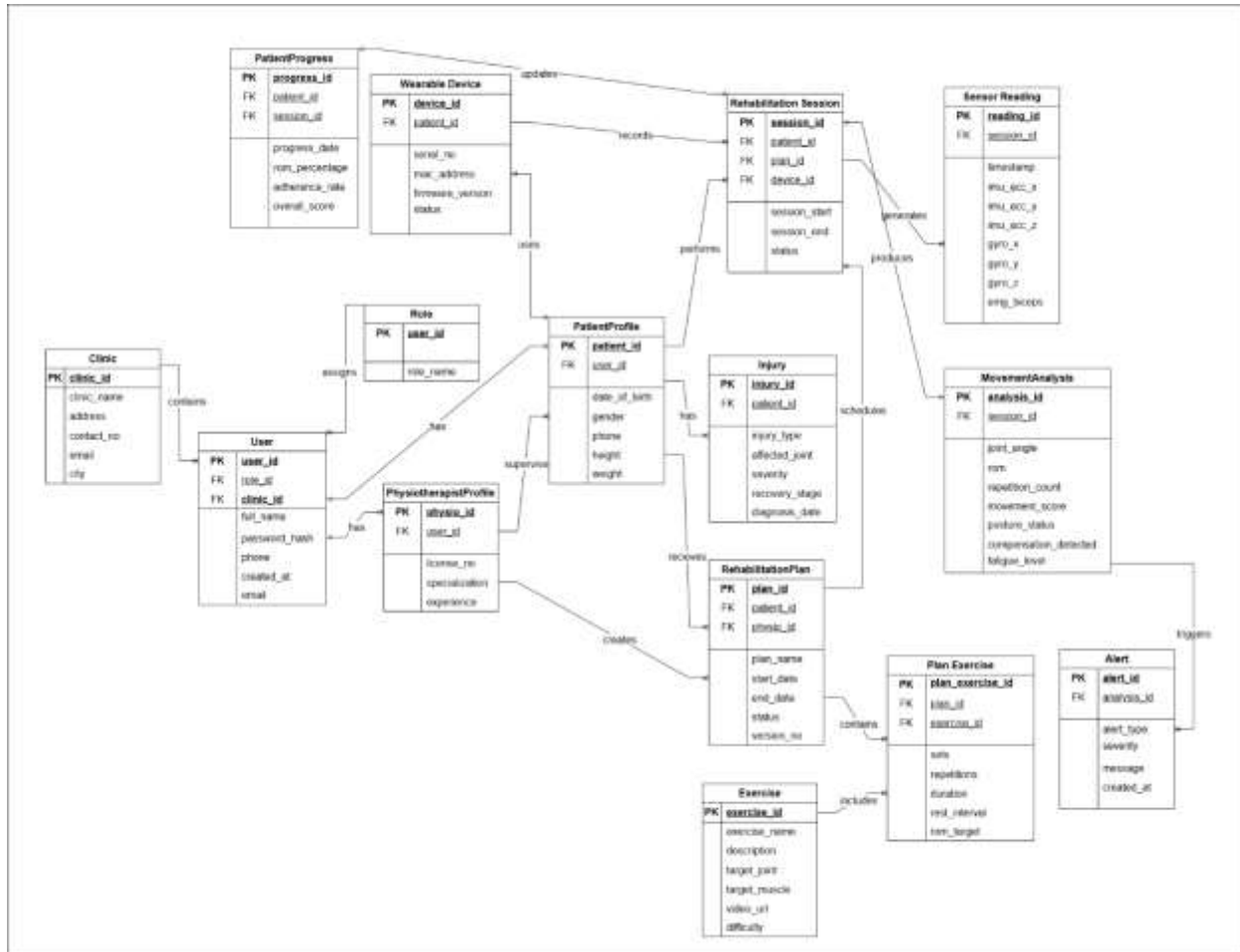


Figure 11 Inteli-Rehab ER Diagram

5.2 DATABASE / STORAGE SOLUTIONS:

Inteli-Rehab uses Supabase (managed PostgreSQL) as its primary relational data store for all account, clinical, protocol, session-summary, alert, progress, and report metadata. Supabase Auth handles authentication and role-based access control for Patient, Physiotherapist, and Clinic Admin accounts, and Supabase Storage holds generated PDF reports.

On the mobile device, a local SQLite cache stores the active protocol, in-progress session data, and any records queued by the offline-first sync engine, so that a live exercise session -- including

movement tracking, digital-twin rendering, and safety alerts -- is never blocked by a lost network connection. High-frequency raw sensor frames are batched locally during a session and summarized before being persisted, rather than being streamed to the cloud row-by-row, to keep bandwidth and storage costs manageable.

5.3 DATA DICTIONARY:

ENTITY: CLINIC

Table 5.1 USER Data Dictionary

Attribute	Type	Description
clinic_id	int	Primary key, unique identifier for the clinic
clinic_name	string	Name of the clinic or hospital
address	string	Physical address of the clinic
contact_no	string	Phone number of the clinic
email	string	Email address of the clinic
city	string	City where the clinic is located

ENTITY: ROLE

Table 5.2 ROLE Data Dictionary

Attribute	Type	Description
user_id	int	Primary key; foreign key to User
role_name	string	Role assigned to the user

ENTITY: USER

Table 5.3 USER Data Dictionary

Attribute	Type	Description
user_id	INT	Primary key, unique identifier for the user
role_id	INT	Foreign key referencing ROLE
clinic_id	INT	Foreign key referencing CLINIC
full_name	VARCHAR	User's full name
password_hash	VARCHAR	Securely hashed password
phone	VARCHAR	User's contact number
created_at	DATETIME	Account creation timestamp

ENTITY: PATIENT

Table 5.4 PATIENT Data Dictionary

Attribute	Type	Description
patient_id	INT	Primary key, unique identifier for the patient
user_id	INT	Foreign key referencing USER
date_of_birth	DATE	Patient's date of birth
gender	VARCHAR	Patient's gender
phone	VARCHAR	Patient's contact number
height	FLOAT	Height in centimeters
weight	FLOAT	Weight in kilograms

ENTITY: PHYSIOTHERAPIST_PROFILE*Table 5.5 PHYSIOTHERAPIST_PROFILE Data Dictionary*

Attribute	Type	Description
physio_id	INT	Primary key, unique identifier for the physiotherapist
user_id	INT	Foreign key referencing USER
license_no	VARCHAR	Professional license number
specialization	VARCHAR	Area of specialization
experience	INT	Years of professional experience
email	VARCHAR	Professional email address

ENTITY: INJURY*Table 5.6 INJURY Data Dictionary*

Attribute	Type	Description
injury_id	INT	Primary key, unique identifier for the injury
patient_id	INT	Foreign key referencing PATIENT_PROFILE
injury_type	VARCHAR	Type of injury
affected_joint	VARCHAR	Joint affected by the injury
severity	VARCHAR	Injury severity level
recovery_stage	VARCHAR	Current stage of recovery
diagnosis_date	DATE	Date of diagnosis

ENTITY: WEARABLE_DEVICE*Table 5.7 WEARABLE_DEVICE Data Dictionary*

Attribute	Type	Description
device_id	INT	Primary key, unique identifier for the wearable device
patient_id	INT	Foreign key referencing PATIENT_PROFILE

serial_no	VARCHAR	Unique serial number
mac_address	VARCHAR	Bluetooth MAC address
firmware_version	VARCHAR	Installed firmware version
status	VARCHAR	Current device status

ENTITY: REHABILITATION_PLAN

Table 5.8 REHABILITATION_PLAN Data Dictionary

Attribute	Type	Description
plan_id	INT	Primary key, unique identifier for the rehabilitation plan
patient_id	INT	Foreign key referencing PATIENT_PROFILE
physio_id	INT	Foreign key referencing PHYSIOTHERAPIST_PROFILE
plan_name	VARCHAR	Rehabilitation plan name
start_date	DATE	Plan start date
end_date	DATE	Plan end date
status	VARCHAR	Current plan status
version_no	INT	Plan version number

ENTITY: EXERCISE

Table 5.9 EXERCISE Data Dictionary

Attribute	Type	Description
exercise_id	INT	Primary key, unique identifier for the exercise
exercise_name	VARCHAR	Exercise name
description	TEXT	Detailed exercise description
target_joint	VARCHAR	Target joint
target_muscle	VARCHAR	Target muscle group

video_url	VARCHAR	Instructional video URL
difficulty	VARCHAR	Exercise difficulty level

ENTITY: PLAN_EXERCISE

Table 5.10 PLAN_EXERCISE Data Dictionary

Attribute	Type	Description
plan_exercise_id	INT	Primary key, unique identifier for the record
plan_id	INT	Foreign key referencing REHABILITATION_PLAN
exercise_id	INT	Foreign key referencing EXERCISE
sets	INT	Number of prescribed sets
repetitions	INT	Number of repetitions
duration	INT	Duration in minutes
rest_interval	INT	Rest interval in seconds
rom_target	FLOAT	Target range of motion (degrees)

ENTITY: REHABILITATION_SESSION

Table 5.11 REHABILITATION_SESSION Data Dictionary

Attribute	Type	Description
session_id	INT	Primary key, unique identifier for the session
patient_id	INT	Foreign key referencing PATIENT_PROFILE
plan_id	INT	Foreign key referencing REHABILITATION_PLAN
device_id	INT	Foreign key referencing WEARABLE_DEVICE
session_start	DATETIME	Session start time
session_end	DATETIME	Session end time
status	VARCHAR	Current session status

ENTITY: SENSOR_READING*Table 5.12 SENSOR_READING Data Dictionary*

Attribute	Type	Description
reading_id	INT	Primary key, unique identifier for the sensor reading
session_id	INT	Foreign key referencing REHABILITATION_SESSION
timestamp	DATETIME	Time the reading was captured
imu_acc_x	FLOAT	Accelerometer X-axis value
imu_acc_y	FLOAT	Accelerometer Y-axis value
imu_acc_z	FLOAT	Accelerometer Z-axis value
gyro_x	FLOAT	Gyroscope X-axis value
gyro_y	FLOAT	Gyroscope Y-axis value
gyro_z	FLOAT	Gyroscope Z-axis value
emg_biceps	FLOAT	EMG reading from biceps

ENTITY: MOVEMENT_ANALYSIS*Table 5.13 MOVEMENT_ANALYSIS Data Dictionary*

Attribute	Type	Description
analysis_id	INT	Primary key, unique identifier for the analysis
session_id	INT	Foreign key referencing REHABILITATION_SESSION
joint_angle	FLOAT	Calculated joint angle (degrees)
rom	FLOAT	Achieved range of motion
repetition_count	INT	Number of completed repetitions
movement_score	FLOAT	Movement quality score (0–100)
posture_status	VARCHAR	Posture assessment
compensation_detected	BOOLEAN	Indicates compensatory movement detection
fatigue_level	VARCHAR	Detected fatigue level

ENTITY: ALERT

Table 5.14 ALERT Data Dictionary

Attribute	Type	Description
alert_id	INT	Primary key, unique identifier for the alert
analysis_id	INT	Foreign key referencing MOVEMENT_ANALYSIS
alert_type	VARCHAR	Alert category
severity	VARCHAR	Alert severity
message	TEXT	Alert description
created_at	DATETIME	Alert generation timestamp

ENTITY: PATIENT_PROGRESS

Table 5.15 PATIENT_PROGRESS Data Dictionary

Attribute	Type	Description
progress_id	INT	Primary key, unique identifier for the progress record
patient_id	INT	Foreign key referencing PATIENT_PROFILE
session_id	INT	Foreign key referencing REHABILITATION_SESSION
progress_date	DATE	Progress record date
rom_percentage	FLOAT	Range of motion improvement percentage
adherence_rate	FLOAT	Exercise adherence percentage
overall_score	FLOAT	Overall rehabilitation progress score

8. ALGORITHM AND IMPLEMENTATION:

This section presents the core algorithms of Inteli-Rehab in Program Design Language (PDL) pseudocode, covering the on-device pipeline from wearable pairing through AI-based safety monitoring to cloud synchronization.

6.1 WEARABLE PAIRING & BIOMETRIC CALIBRATION ALGORITHM:

This algorithm handles BLE pairing with the wearable band and establishes the patient's biometric baseline (neutral joint position and resting muscle level) used to normalize every later reading.

```
pairAndCalibrate(patientID, deviceMAC):
```

```
    device = scanForBLEDevice(deviceMAC)
```

```
    IF device NOT FOUND THEN
```

```
        RETURN error "Device not found"
```

```
    ENDIF
```

```
    connection = connectBLE(device)
```

```
    IF connection.status != "connected" THEN
```

```
        RETURN error "Pairing failed"
```

```
    ENDIF
```

```
    registerDevice(patientID, device.macAddress)
```

```
    displayInstruction("Hold arm still in neutral position")
```

```
    neutralSamples = collectIMUSamples(duration = 5s)
```

```
    neutralAngle = average(neutralSamples)
```

```
    displayInstruction("Relax muscles")
```

```
    restingEMGSamples = collectEMGSamples(duration = 5s)
```

```
    restingEMGLevel = average(restingEMGSamples)
```

```

IF signalQuality(neutralSamples, restingEMGSamples) < THRESHOLD THEN
    RETURN error "Poor signal quality, please reposition sensors"
ENDIF

```

```

baseline = saveBiometricBaseline(patientID, neutralAngle, restingEMGLevel)

RETURN baseline

```

6.2 SENSOR FUSION ALGORITHM:

This algorithm fuses the two IMUs into a single relative joint angle and uses that angle stream to count repetitions and track range-of-motion progress against the assigned protocol.

```

computeJointAngle(imu1Reading, imu2Reading, baseline):

```

```

    q1 = madgwickUpdate(imu1Reading.accel, imu1Reading.gyro, imu1Reading.mag)

```

```

    q2 = madgwickUpdate(imu2Reading.accel, imu2Reading.gyro, imu2Reading.mag)

```

```

    relativeOrientation = quaternionDifference(q1, q2)

```

```

    rawAngle = quaternionToEulerAngle(relativeOrientation)

```

```

    jointAngle = rawAngle - baseline.neutralJointAngle

```

```

    RETURN jointAngle

```

```

trackRepetition(session, jointAngle, protocol):

```

```

    IF session.state == "extending" AND jointAngle >= EXTENSION_THRESHOLD
    THEN

```

```

        session.state = "flexing"

```

```

    ELSE IF session.state == "flexing" AND jointAngle <= FLEXION_THRESHOLD
    THEN

```

```

        session.repsCompleted += 1

```

```

    session.state = "extending"

ENDIF

session.currentROM = max(session.currentROM, jointAngle)

session.romPercentage = (session.currentROM / protocol.targetROM) * 100

RETURN session

```

6.3 EMG SIGNAL PROCESSING ALGORITHM:

This algorithm converts raw biceps/triceps EMG into a normalized activation level and a spectral fatigue index.

```

processEMGSignal(rawEMG, baseline, initialMedianFrequency):

    filtered = bandpassFilter(rawEMG, low = 20Hz, high = 450Hz)

    rectified = fullWaveRectify(filtered)

    envelope = movingRMS(rectified, window = 100ms)

    activationLevel = (envelope - baseline.restingEMGLevel) / MVC_REFERENCE

    activationLevel = clamp(activationLevel, 0, 1)

    spectrum = fastFourierTransform(filtered)

    medianFrequency = computeMedianFrequency(spectrum)

    fatigueIndex = (initialMedianFrequency - medianFrequency) / initialMedianFrequency

    RETURN activationLevel, fatigueIndex

```

6.4 MOVEMENT ANOMALY DETECTION ALGORITHM:

This algorithm flags compensatory or incorrect movement patterns by measuring how well an autoencoder trained on correct-form data can reconstruct the current movement.

```

detectMovementAnomaly(featureVector):

    // featureVector = [jointAngle, angularVelocity, activationLevel, ROM]

    normalizedInput = normalize(featureVector)

    reconstructed = autoencoder.forward(normalizedInput)

    reconstructionError = meanSquaredError(normalizedInput, reconstructed)

    IF reconstructionError > ANOMALY_THRESHOLD THEN

        severity = classifySeverity(reconstructionError)

        RETURN true, severity

    ELSE

        RETURN false, "none"

    ENDIF

```

6.5 FATIGUE TREND DETECTION & ALERT ALGORITHM:

This algorithm looks at the recent temporal sequence of fatigue-index readings to detect a developing fatigue trend before it becomes unsafe.

```

detectFatigueTrend(sessionWindow):

    // sessionWindow = sequence of last N fatigue-index readings

    sequence = normalizeSequence(sessionWindow)

    prediction = lstmModel.predict(sequence)

    IF prediction.stage == "moderate" THEN

        triggerAlert("fatigue_warning", severity = "mild")

    ELSE IF prediction.stage == "critical" THEN

        triggerAlert("fatigue_warning", severity = "critical")

```

```
    recommendSessionPause()
```

```
ENDIF
```

```
RETURN prediction.stage
```

6.6 SESSION REPORT GENERATION & CLOUD SYNC ALGORITHM:

This algorithm consolidates a completed session's metrics and alerts into a report and reliably syncs it using the offline-first queue.

```
generateSessionReport(session):
```

```
    report = initializeReport(session.patientID)
```

```
    report.addMetric("Repetitions", session.repsCompleted)
```

```
    report.addMetric("ROM Progress", session.romPercentage)
```

```
    report.addMetric("Consistency Score", session.consistencyScore)
```

```
    alerts = getAlertsForSession(session.sessionID)
```

```
    FOR EACH alert IN alerts:
```

```
        report.addSection("Alert", alert)
```

```
    ENDFOR
```

```
    badges = evaluateNewBadges(session)
```

```
    report.setBadges(badges)
```

```
    reportID = saveReportLocally(report)
```

```
    IF networkAvailable() THEN
```

```
        syncToCloud(session, report)
```

ELSE

 queueForSync(session, report)

ENDIF

RETURN reportID

7. Software requirements traceability matrix

The table below maps each functional requirement identified for Inteli-Rehab to the design component(s) responsible for realizing it. Requirement numbering follows the module breakdown defined in the project's Scope Document; it should be reconciled with the formal SRS requirement IDs once that document is finalized.

Table 7.1 Requirements Traceability Matrix

Req #	Referenced Item	Design Component	Component Item(s)
FR-1	Patient Registration & Clinic/Physiotherapist Selection	Class Diagram	Patient class, selectClinic method, OnboardingController
FR-2	User Login (Patient / Physiotherapist / Admin)	Sequence Diagram	login method, AuthService
FR-3	Physiotherapist Account Creation	Class Diagram	ClinicAdmin class, addPhysiotherapist method
FR-4	Maintain Clinical Exercise Database	Class Diagram	Exercise class, ExerciseRepository
FR-5	Assign Personalized Exercise Protocol	Class Diagram	ExerciseProtocol class, assign method
FR-6	Pair Wearable Device via BLE	Sequence Diagram	BLE Client Manager, pairAndCalibrate method

Req #	Referenced Item	Design Component	Component Item(s)
FR-7	Perform Biometric Calibration	Sequence Diagram	BiometricBaseline class, calibrate method
FR-8	Start & Track Exercise Session	Sequence Diagram	ExerciseSession class, start, countRepetition methods
FR-9	Compute Joint Angles & Muscle Activation	Class Diagram	SensorFusionEngine, EMGProcessor classes
FR-10	Render Live 3D Digital Twin	Sequence Diagram	DigitalTwinRenderer class
FR-11	Detect Movement Anomaly & Trigger Corrective Alert	Sequence Diagram	AIInferenceEngine (Autoencoder), AlertManager
FR-12	Detect Fatigue Trend & Trigger Warning	Sequence Diagram	AIInferenceEngine (LSTM), AlertManager
FR-13	View Progress, ROM Charts & Gamification	Class Diagram	ProgressRecord, Badge classes
FR-14	Monitor Patient Progress Dashboard	Sequence Diagram	Physiotherapist Web Dashboard, getPatientProgress
FR-15	Adjust Protocol & Approve Progression	Class Diagram	ExerciseProtocol class, adjust, approveProgression
FR-16	Generate PDF Report	Sequence Diagram	Report class, requestReport method
FR-17	Sync Session Data to Cloud (Offline-First)	Sequence Diagram	Offline-First Sync Queue, Cloud Backend API

Req #	Referenced Item	Design Component	Component Item(s)
FR-18	Manage Physiotherapist Accounts (Clinic Panel)	Class Diagram	ClinicAdmin class, Clinic Admin Panel

8. Human interface design

IntelliRehab provides a simple and user-friendly interface designed for patients and physiotherapists to interact with the rehabilitation system efficiently.

Patients can log in, view assigned rehabilitation exercises, connect their wearable device, and perform exercises while receiving real-time feedback. The application displays movement progress, exercise performance, sensor status, and rehabilitation history through an interactive interface. Patients can also view assessment results and recommendations generated from their exercise sessions.

Physiotherapists can access patient records, assign personalized exercise plans, monitor rehabilitation sessions, and review performance reports. The interface presents sensor-based analysis, movement quality scores, and progress information to help physiotherapists evaluate recovery and make informed decisions.

The system provides feedback through visual indicators, progress reports, alerts, and exercise recommendations, allowing users to easily understand rehabilitation status and take appropriate actions.

8.1. SCREEN IMAGES FOR MOBILE APP:

These screens represent the core user interface of the Intelli-Rehab system, showing how users interact with key features such as Registration, Rehabilitation session, Progress Checking, session history viewing, Web Portal landing page, and administrative functions. The design ensures a consistent and intuitive experience across the web and mobile application. Each interface focuses on clarity, ease of use, and smooth access to primary system functions.

The Intelli-Rehab system includes:

- Intelli-Rehab Web Portal Landing Page
- Web Login Page & Web Dashboard for clinic and physiotherapist
- Login/Registration Page for Patient Mobile Application
- Patient Dashboard
- Patient Sensor Connection and Calibration Screens
- Exercise Review Screen

- Rehabilitation Session Summary Screen

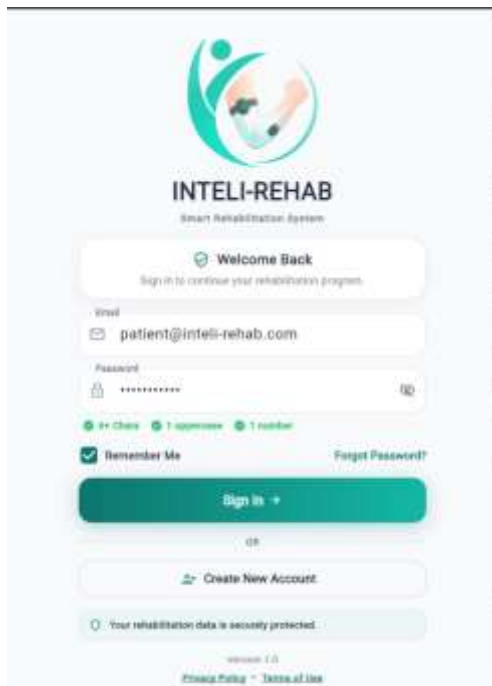


Figure 12 Sign in Screen

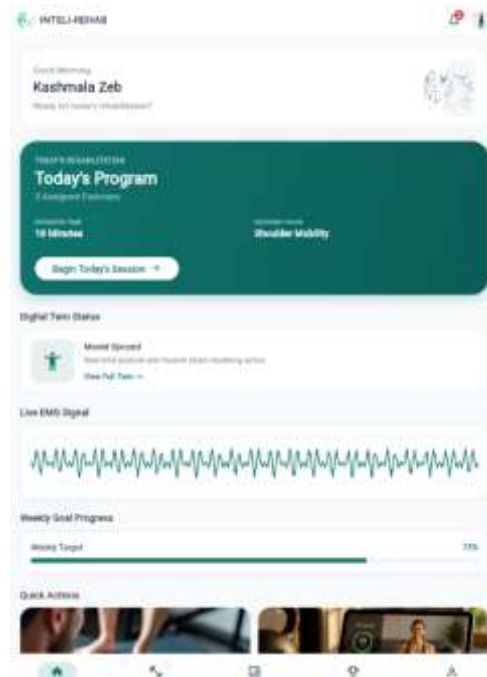


Figure 13 Patient Dashboard

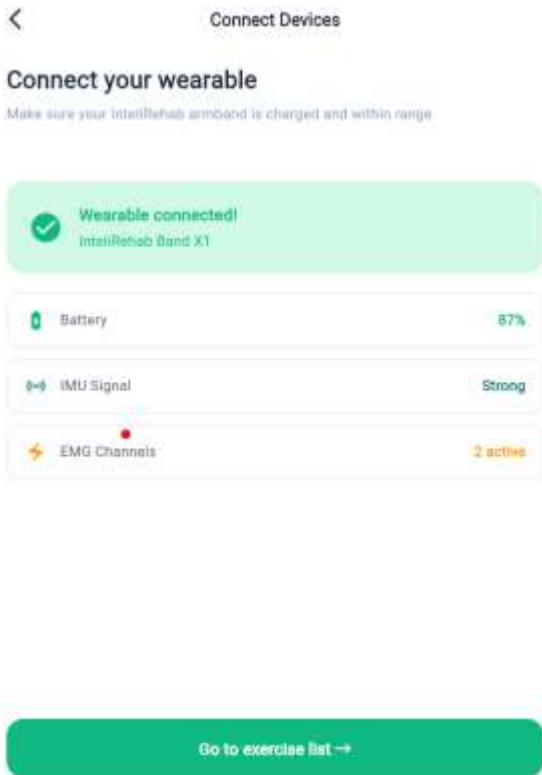


Figure 14 Wearable Connection Screen



Figure 15 Exercise Preview Screen



Figure 16 Rehabilitation Session Summary Screen

Screen objects and actions for Patient Mobile App

On the Website the patients interact with several screen objects that perform specific actions, allowing smooth and intuitive navigation. These objects can be categorized into buttons, text fields, image displays, and alerts. Each screen object has an associated action that drives the system's functionality.

1. Sign In

- **Screen Object:** Email field, password field with toggle, password strength indicators, Remember Me checkbox, Sign In button, Create New Account button
- **Action:** Enter credentials to sign in, toggle password visibility, and navigate to account creation

2. Sign Up

- **Screen Object:** Registration form fields, password strength indicators, terms checkbox, Create Account button
- **Action:** Register a new account by filling in details, meeting password requirements, and agreeing to terms

3. Connect Devices

- **Screen Object:** Connection status banner, device metrics (Battery, IMU Signal, EMG Channels), Go to exercise list button
- **Action:** Verify wearable connection, check device health, and proceed to exercises

4. Exercise Detail

- **Screen Object:** Exercise video/image, duration timer with controls, instructions, focus area tags, muscle diagram, Start Session button
- **Action:** Review exercise details, adjust duration, and begin the session

5. Session Summary

- **Screen Object:** Performance score, session statistics grid (Accuracy, ROM, Time, Reps, Muscle Activation), progress chart, ROM gauge, muscle activity bars
- **Action:** View session results, track recovery progress, and analyze performance metrics

8.2. SCREEN IMAGES FOR WEB PORTAL:

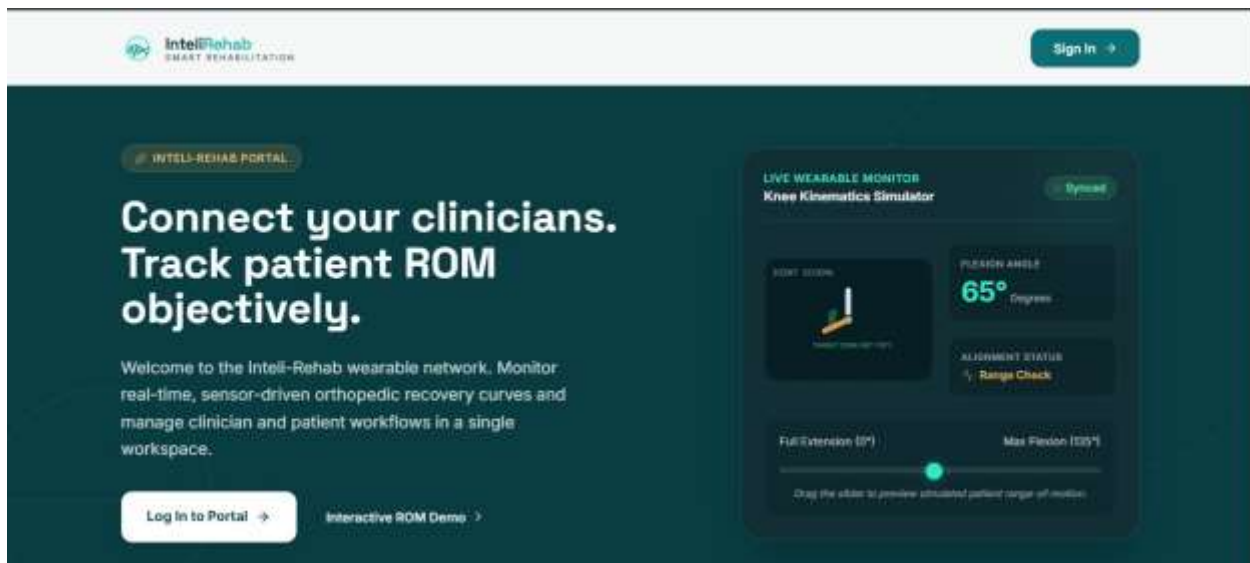


Figure 17 Web Portal Landing Page

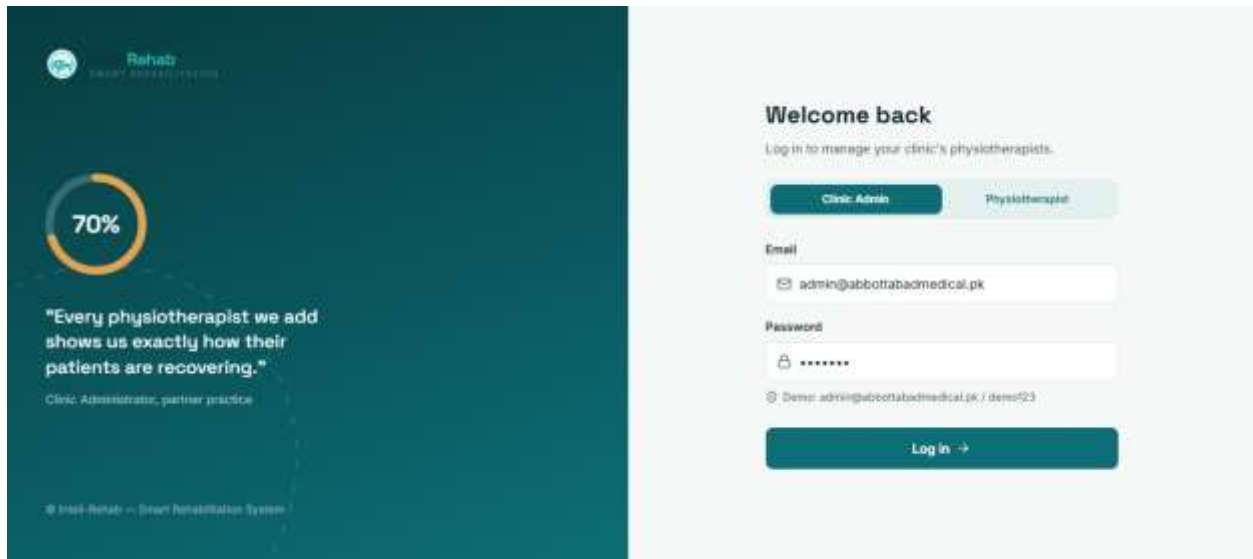


Figure 18 Role Based Login Page

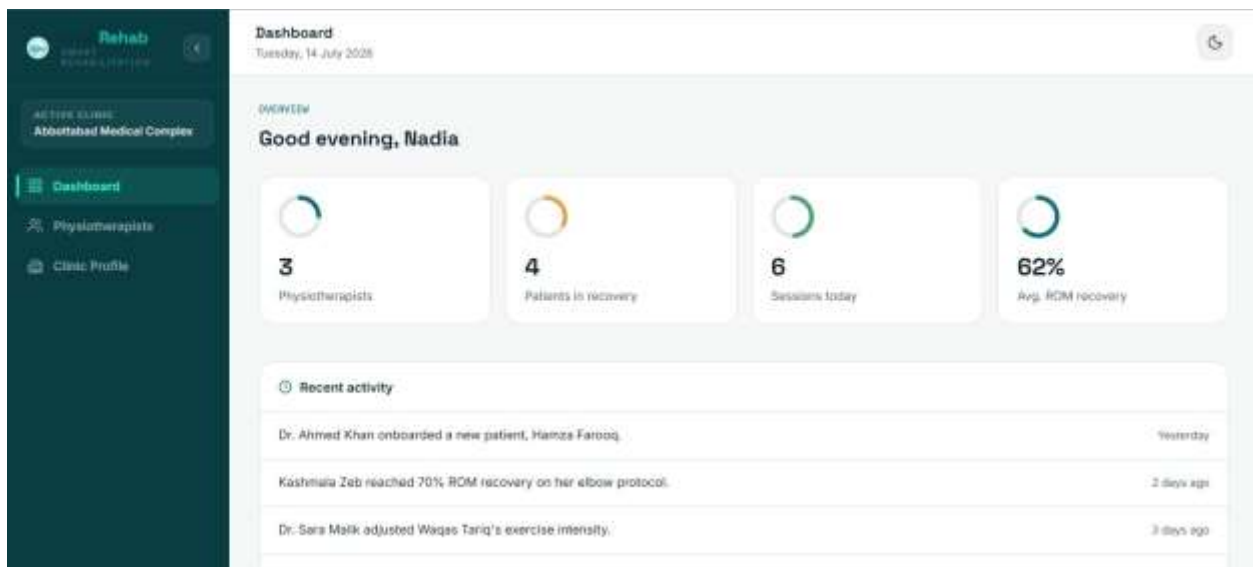


Figure 19 Clinic Administrator's Dashboard

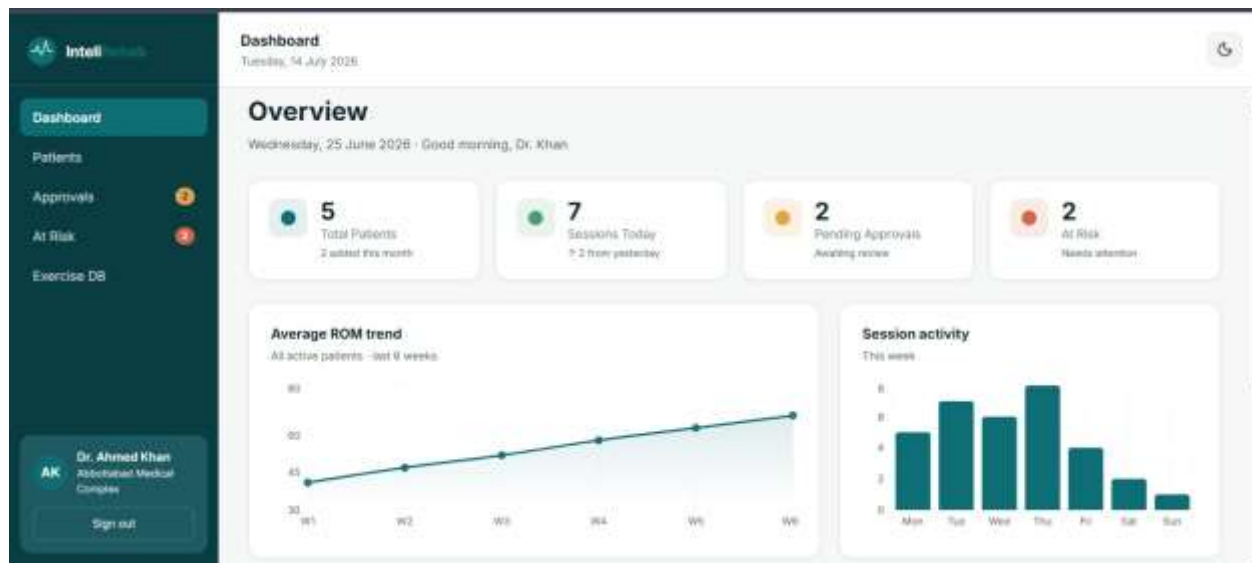


Figure 20 Physiotherapist Dashboard

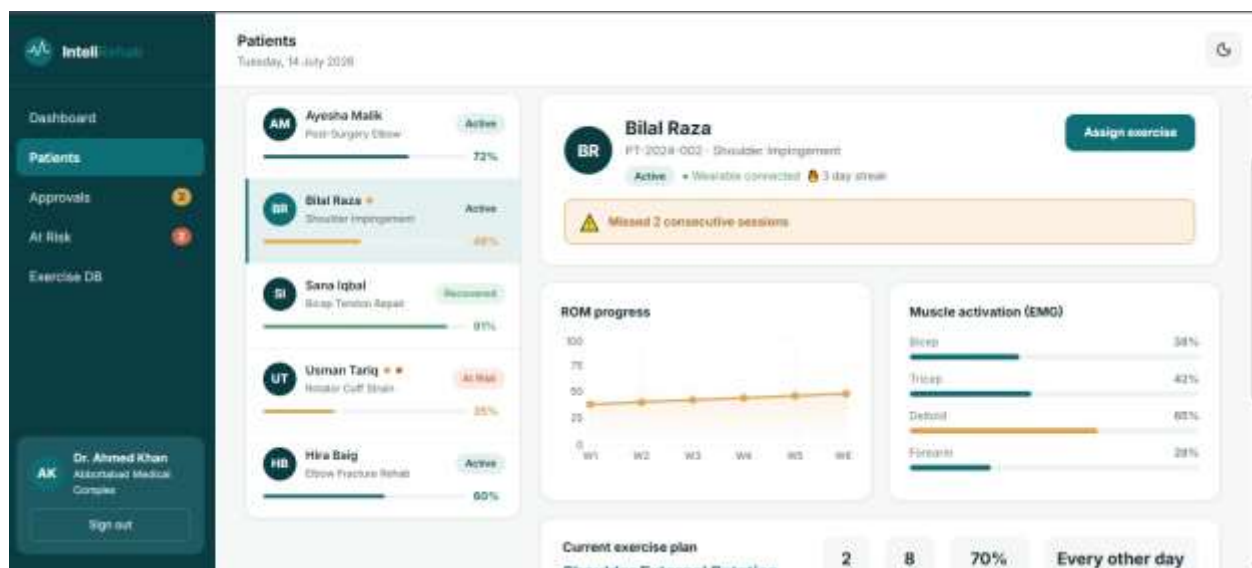


Figure 21 Patients Information Display Page

Screen objects and actions for Clinic Admin & Physiotherapist Portal

1. Landing Page

- Screen Object:** Hero section, ROM simulator panel, Sign In button, Interactive ROM Demo link
- Action:** Preview simulated patient ROM and navigate to portal login

2. Portal Sign In

- **Screen Object:** Role toggle, email field, password field, Log In button
- **Action:** Select role and sign in to manage clinic workflows

3. Clinic Admin Dashboard

- **Screen Object:** Stat cards, recent activity feed, sidebar navigation
- **Action:** View clinic metrics and track recent activity

4. Physiotherapist Dashboard

- **Screen Object:** Stat cards, ROM trend chart, session activity chart, sidebar navigation
- **Action:** Monitor patients, review approvals, identify at-risk cases, and track recovery trends

5. Patients Management

- **Screen Object:** Patient list, detail panel, ROM progress chart, EMG chart, Assign exercise button
- **Action:** View patient progress, check muscle activation data, and assign exercises

9. Appendix I

The following external resources and tools were referenced during the design of Inteli-Rehab:

Table 9.1 Appendix

Resource	Link
Flutter Documentation	https://docs.flutter.dev/
Unity Documentation	https://docs.unity3d.com/
Three.js Documentation	https://threejs.org/docs/
Supabase Documentation	https://supabase.com/docs
TensorFlow Lite (On-Device Inference)	https://www.tensorflow.org/lite
PyTorch Documentation	https://pytorch.org/docs/

Resource	Link
Bluetooth Low Energy (BLE) Specifications	https://www.bluetooth.com/specifications/specs/
React.js Documentation	https://react.dev/
PostgreSQL Documentation	https://www.postgresql.org/docs/
UML Reference -- StarUML	https://staruml.io/
UML / Diagramming Reference -- diagrams.net	https://www.diagrams.net/
Incremental Modeling Guide	https://agilemodeling.com/essays/bestpractices.htm